
AdaMeZO: Adam-style Zeroth-Order Optimizer for LLM Fine-tuning Without Maintaining the Moments

Zhijie Cai^{* 1 2 3} Haolong Chen^{* 1 2 3} Guangxu Zhu^{1 2 3 4}

Abstract

Fine-tuning LLMs is necessary for various dedicated downstream tasks, but classic backpropagation-based fine-tuning methods require substantial GPU memory. To this end, a recent work, MeZO, which relies solely on forward passes to fine-tune LLMs, significantly reduces GPU requirements at the cost of slower convergence due to its indifference to loss landscapes. Standard solutions, such as Adam, explore loss landscapes by estimating the first- and second-order moments and storing them in memory to guide the model’s movement through dimensions with lower curvature and vice versa. However, directly applying Adam negates MeZO’s advantage as it will triple the memory requirement. In light of this, we propose AdaMeZO, a zeroth-order optimizer that leverages Adam-style first- and second-moment estimates without maintaining them in memory. We present a theoretical analysis of AdaMeZO, corroborated by extensive experiments demonstrating AdaMeZO’s performance, showing that AdaMeZO can outperform MeZO while requiring up to 70% fewer forward passes. Trajectory visualizations affirm AdaMeZO’s ability to adapt to diverse loss landscapes.

1. Introduction

Fine-tuning LLMs is necessary for specialized downstream tasks and has recently attracted significant attention. Many works have emerged that aim to tune models while accessing as little memory as possible. Popular first-order meth-

ods known as parameter-efficient fine-tuning (PEFT) to alleviate the heavy memory cost by modifying only a small (potentially extra) part of the whole model (Hu et al., 2022; Li & Liang, 2021; Lester et al., 2021; Dettmers et al., 2023; Pan et al., 2024). Additionally, a zeroth-order method (Malladi et al., 2023) enables discarding backpropagation, the primary contributor to LLM fine-tuning’s memory cost, making it accessible on resource-limited devices.

As shown in Table 1, MeZO features an SGD-style update rule (Rumelhart et al., 1986; Bottou et al., 2018), allowing in-place parameter modification. After in-place model perturbation for gradient projection estimation, the gradients are not dumped into memory; instead, they are generated by a pseudo-random number generator (PRNG) before being scaled by the previously computed projection, reducing the memory cost for fine-tuning to the equivalent of deploying one. However, updating the model with only the most recent gradient estimate can lead to slower convergence, especially with noisy, isotropic zeroth-order gradient estimators. In comparison, adaptive optimizers like Adam (Kingma & Ba, 2014) and AdamW (Loshchilov & Hutter, 2017), which correct updates with preconditioners, are more widely adopted since the loss landscapes of LLMs exhibit complex curvature spectra across different dimensions, as documented in (Sagun et al., 2016; Ghorbani et al., 2019; Zhang et al., 2023; Das et al., 2024).

However, adaptive optimizers retain historical gradient information in the memory. In the case of Adam, the first and second moments are the accumulated gradients and the quadratic gradients. In other words, two vectors of the same size as the model need to be kept in memory. Considering that first-order methods use backpropagation, the additional memory cost is relatively small. But in the context of zeroth-order optimizers, the memory cost is multiplied.

Adaptive zeroth-order optimizers for LLM fine-tuning have recently attracted research interest, as shown in Table 1. Pioneering works include HiZOO (Zhao et al., 2024b), ZO-AdaMU (Jiang et al., 2024), and Helene (Zhao et al., 2024a). HiZOO proposes approximating the diagonal Hessian using an additional forward-pass oracle, which doubles the memory required to store it. Helene is a more direct integration of zeroth-order gradient estimation and

^{*}Equal contribution ¹Shenzhen International Center for Industrial and Applied Mathematics ²Shenzhen Research Institute of Big Data ³The Chinese University of Hong Kong-Shenzhen ⁴Shenzhen Loop Area Institute. Correspondence to: Guangxu Zhu <gxzhu@sribd.cn>.

Table 1. Key features for AdaMeZO and methods in comparison. P in the first column denotes the amount of memory required to store the model weight, and $B \gg P$ denotes the amount of memory required to perform backpropagation. $\delta \ll 1$ is a small positive number. “FP” abbreviates forward pass.

	Param. memory	FP per step	1st moment	2nd moment
Adam (Kingma & Ba, 2014)	$3P + B$	1	✓	✓
MeZO (Malladi et al., 2023)	P	2	×	×
HELENE (Zhao et al., 2024a)	$3P$	2	✓	✓
HiZO (Zhao et al., 2024b)	$2P$	3	×	✓
AdaMeZO	$(1 + \delta)\bar{P}$	$\frac{2}{\delta}$	✓	✓

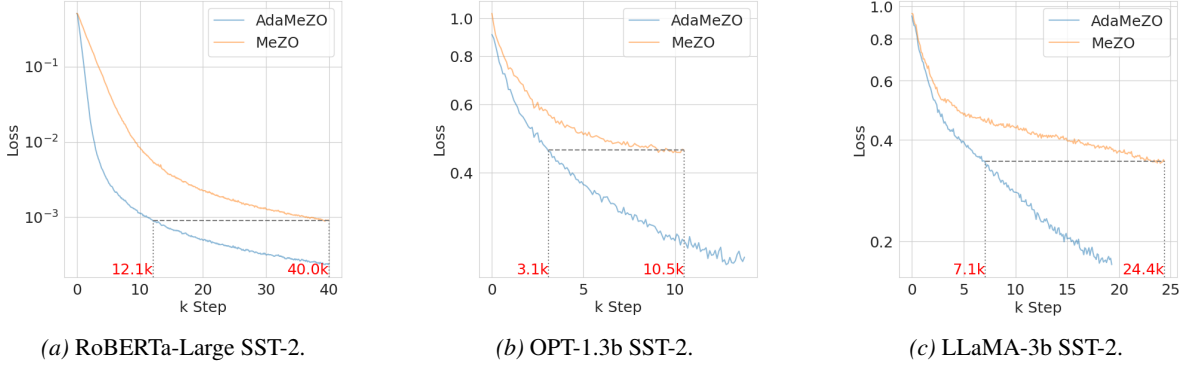


Figure 1. Loss curves of MeZO and AdaMeZO on the SST2 task. When fine-tuning RoBERTa-large, OPT-1.3b, LLaMA-3b, AdaMeZO took 69.75%, 70.48%, 70.90% fewer forward passes to reach the loss values of MeZO at terminations, respectively. Hyperparameters and terminal conditions are detailed in Section B.4.

an Adam optimizer, and ZO-AdaMU replaces the moments with an uncertain version. As a result, the memory requirement is tripled to store both diagonal Hessian estimation and cumulative history gradients. However, despite the substantial increase in memory cost, they still use much less memory than first-order approaches and achieve a noticeable performance gain over MeZO.

In light of the above, we introduce AdaMeZO¹, a zeroth-order optimizer that leverages Adam-style first- and second-moment estimates to accelerate convergence without requiring additional memory to store them. This is made possible by 1) computing truncated moments that discard outdated gradients rather than faithfully maintaining the full moment estimations, and 2) block-wise generation of random gradient direction with a finer operation of the PRNG. As a result, AdaMeZO significantly reduces the number of forward passes required for convergence and improves the fine-tuned model’s performance. A summary of the contributions of this work is as follows.

1. We introduce AdaMeZO, an optimizer that uses zeroth-order gradient estimates and updates with Adam-style first- and second-moment estimates. Although the moments are necessary to compute the

model updates, with truncated approximations and finer PRNG operations, they do not need to be stored in memory. In this way, AdaMeZO can theoretically use no additional memory to improve convergence with preconditioning.

2. We establish a convergence bound of AdaMeZO under a non-convex assumption that recovers the convergence rate of preconditioned MeZO with multiples of memory cost.
3. We conduct extensive experiments to evaluate AdaMeZO’s performance of AdaMeZO. We first employ 2-dimensional toy functions and visualize the trajectories of optimization. They demonstrate that AdaMeZO converges to optimal points, whereas MeZO does not, even with the same step budgets. Then we demonstrate AdaMeZO’s performance by fine-tuning different models (RoBERTa (Liu et al., 2019b), OPT (Zhang et al., 2022a), and LLaMa (Touvron et al., 2023)) for a task set identical to MeZO’s. It is found that AdaMeZO almost always reaches the same termination condition as MeZO, with up to 70% fewer forward passes and higher performance.

¹Codes are available at <https://github.com/shawnnn3di/AdaMeZO>.

2. Related Works

2.1. Zeroth-order Optimizers for LLMs

Zeroth-order optimization is also known as derivative-free or black-box optimization. Previously, it was used for situations where the objective function has no derivatives or obtaining derivatives is expensive. Fine-tuning LLMs falls into the latter case and sometimes both for non-differentiable objectives (Tang et al., 2023; Zhang et al., 2024c). In the context of modern deep learning, it translates to the emission of auto-differentiation by backward propagation (Rumelhart et al., 1986), resulting in hugely reduced memory consumption. Some past work on zeroth-order optimizers include (Spall, 1992; 1997; Vakhitov et al., 2009; Agarwal et al., 2009; Raginsky & Rakhlin, 2011; Jamieson et al., 2012; Wang et al., 2020; FairScale authors, 2021). MeZO (Malladi et al., 2023) firstly adopts the classical SPSA (Spall, 1992) to fine-tune billion-level dimension LLMs based on low rank assumptions on LLM fine-tuning, achieving comparable performance with much fewer GPU hours. A survey on concurrent extensions on top of MeZO can be found in Section A. Notably, from-scratch zeroth-order optimization on smaller networks is also of great research interest (Chen et al., 2023).

2.2. First-order Optimizers for LLMs

First-order optimization algorithms form the backbone of training or fine-tuning LLMs, offering computational efficiency and scalability across billions of parameters. One of the most classic solutions is Adam (Kingma & Ba, 2014), which updates based on first- and second-order moments. Some of its variants are as follows. AdamW (Loshchilov & Hutter, 2017) introduces adaptive learning rates via moment estimates, achieving faster convergence on nonconvex objectives. LAMB (You et al., 2019) features a layer-wise adaptation strategy to accelerate the training of large models employing large batches. Adafactor (Shazeer & Stern, 2018) reduces memory usage by maintaining factored second-moment estimates rather than the faithful estimates. AdaBelief (Zhuang et al., 2020) replaces Adam’s second moment estimates with a squared gradient with the squared difference between the gradient and its running mean to improve convergence and generalization. Lion (Chen et al., 2022) uses only sign-based moment updates without per-parameter scaling to reduce memory costs. Adabound (Luo et al., 2019) stabilizes learning rates between dynamic lower and upper thresholds to transition from adaptive behavior to SGD-like stability. RAdam (Liu et al., 2019a) introduces rectification to stabilize adaptive learning rates, improving training stability in the early iterations. (Defazio et al., 2024) introduced a schedule-free optimizer that requires no additional hyperparameters beyond those of standard optimizers with momentum. In-

terestingly, (Zhang et al., 2024b) finds that block structures of diagonal Hessians can help reduce memory costs without harming performance. All of these variants feature empirical estimations of first and second moments, but with changes such as moment centering and regularization, which implies that the proposed method can also be applied to the zeroth-order versions of these variants. Additionally, (Pethick et al., 2025) explores leveraging linear minimization oracles to adapt to loss landscapes without Adam-style updates.

2.3. Acceleration by Adam

Compared with first-order optimizers, second-order informed optimizers incorporate second-order information during gradient calculation. The design of Adam mimics Newton’s method with second derivatives. Specifically, the second moment can be viewed as a rough approximation to the inverse Hessian. Lines of work provide analytic or numeric support for Adam’s near-diagonal Hessians estimation in deep learning. (Das et al., 2024) formalizes that diagonally-dominant Hessians make Adam mathematically faster. (Zhang et al., 2024a) finds block-diagonal Hessians in real neural networks and shows Adam outperforms SGD precisely due to this structure. Empirically, (El-sayed et al., 2024) measures strong diagonal dominance in MLP Hessians. (Gui et al., 2021) demonstrates that over-parameterization further drives the Hessian toward a diagonal form. Interestingly, (Ghorbani et al., 2019) found that the Hessian spectra of deep neural networks become stable after less than 1% training step budget. (Kunstner et al., 2023) finds that Adam’s great performance could be attributed to its similarity to sign descent with momenta.

3. Methods

In this section, we first introduce the classic forward-pass-only gradient estimator, SPSA, which is the foundation of MeZO (Malladi et al., 2023). Then, we will explain why direct splicing of SPSA with the Adam-style update rule leads to excessive memory usage and how our technique can prevent this.

3.1. Preliminaries

Definition 3.1 (Simultaneous Perturbation Stochastic Approximation, SPSA (Spall, 1992)). *Given a model with weight $\mathbf{w}_t$ at step t and objective function $\mathcal{L}$, SPSA estimates the gradient on a batch $\mathcal{B}$ with perturbation scale $\mu > 0$ and random direction $\mathbf{z}_t$ as*

$$\mathbf{g}_t = \frac{\mathcal{L}(\mathbf{w}_{t-1} + \mu \mathbf{z}_t, \mathcal{B}_t) - \mathcal{L}(\mathbf{w}_{t-1} - \mu \mathbf{z}_t, \mathcal{B}_t)}{2\mu} \mathbf{z}_t. \quad (1)$$

Following prior works, we assume $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$. It can

Algorithm 1 *h*-MeZO

Input: Initialized model parameters $\mathbf{w}_0 \in \mathbb{R}^d$, loss function $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$, step budget T , perturbation scale μ , learning rate η , horizon h , first EMA ratio β_1

Output: Trained model parameters $\mathbf{w}_T$

seeds, projs $\leftarrow []$, []

for $t = 1, \dots, T$ **do**

Sample batch $\mathcal{B}_t$ and random seed s

Reset the PRNG with random seed s , spawn $\mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, I_d)$

Estimate p_t using Equation (1) # *in-place model perturbation*

seeds.append(s), projs.append(p_t)

$\mathbf{w}_t \leftarrow \mathbf{w}_t$

for $\tau = 1, \dots, h$ **do**

$p \leftarrow \text{projs}[-\tau]$, $s \leftarrow \text{seeds}[-\tau]$

Reset the PRNG with random seed s , spawn $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I_d)$

$\mathbf{w}_t \leftarrow \mathbf{w}_t - \eta \beta_1^{\tau-1} p \mathbf{z}$

end for

end for

be shown that $\mathbf{g}_t \rightarrow \nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t)$ as $\mu \rightarrow 0$, and is treated as an unbiased gradient estimator with a sufficiently small perturbation scale μ . With an SGD-styled update rule of $\mathbf{w}_t \leftarrow \mathbf{w}_t - \eta \mathbf{g}_t$, modifying the model parameters for gradient estimation and model update can be done in-place. MeZO runs quickly on GPUs since they can spawn random gradients the size of 77.5 B within a second². As a result, MeZO generates less information in memory during fine-tuning than backpropagation. The method is shown to yield competitive performance.

3.2. First Moment Can Be Recovered Without Additional Memory

Using the first moment computed by history gradients with EMA updates is a widely used technique to cancel out instantaneous gradient noise, thereby promoting convergence. Common first-order algorithms require an additional trunk of memory of size P to store the current first moment $\mathbf{m}_t$ as follows:

$$\mathbf{m}_t \leftarrow \beta_1 \mathbf{m}_t + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{w}_t \leftarrow \mathbf{w}_t - \eta \mathbf{m}_t.$$

However, the MeZO-style in-place parameter update allows the first moment to be approximated without storing history gradients. Specifically, we unroll the recursion into independent gradients, set a hyperparameter, the horizon h , and discard the outdated gradients computed more than h steps ago, then employ a similar in-place parameter update process as in MeZO as Equation (2) and detailed in Algo-

rithm 1.

$$\mathbf{m}_t \approx (1 - \beta_1)(\mathbf{g}_t + \beta_1 \mathbf{g}_{t-1} + \dots + \beta_1^{t-h-1} \mathbf{g}_{t-h-1}). \quad (2)$$

Remark 3.2. The idea behind Algorithm 1 is that the share of a history gradient $\mathbf{g}_{t-t'}$ decays quickly. As an example, after sufficiently long steps, the share of $\mathbf{g}_{t-10}$ approximates $0.9^{10}/(1/(1-0.9)) \approx 0.0387$ at $\beta_1 = 0.9$. It implies that the key components of the first moment are several of the most recent gradients, while the rest are relatively safe to be omitted. Supported by the PRNG as a coder of the random gradients, Algorithm 1 can use truncated first moments without additional memory.

3.3. Second Moment Informed Updates Without Additional Memory

We can similarly recover a truncated second moment. However, bringing them into the update will still be memory-intensive. We investigate the issue and present our solution in this subsection.

An Adam-style update rule can be expressed as follows:

$$\begin{aligned} \mathbf{m}_t &\leftarrow \beta_1 \mathbf{m}_t + (1 - \beta_1) \mathbf{g}_t, \\ \mathbf{v}_t &\leftarrow \beta_2 \mathbf{v}_t + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t, \\ \mathbf{w}_t &\leftarrow \mathbf{w}_t - \eta \frac{\mathbf{m}_t}{\sqrt{\mathbf{v}_t + \epsilon}}. \end{aligned}$$

Unlike Equation (2), we can't decompose the updates into independent gradients. Instead, we will get a summation of preconditioned gradients as follows:

$$\begin{aligned} \frac{\mathbf{m}_t}{\sqrt{\mathbf{v}_t + \epsilon}} &\approx (1 - \beta_1) \left(\frac{\mathbf{g}_t}{\sqrt{\mathbf{v}_t + \epsilon}} + \frac{\beta_1 \mathbf{g}_{t-1}}{\sqrt{\mathbf{v}_t + \epsilon}} + \dots \right. \\ &\quad \left. + \frac{\beta_1^{t-h-1} \mathbf{g}_{t-h-1}}{\sqrt{\mathbf{v}_t + \epsilon}} \right), \end{aligned}$$

with a common conditioner $\sqrt{\mathbf{v}_t + \epsilon}$ that can only be recovered with multiple gradients as follows:

$$\begin{aligned} \mathbf{v}_t &\approx (1 - \beta_2) (\mathbf{g}_t \odot \mathbf{g}_t + \beta_2 \mathbf{g}_{t-1} \odot \mathbf{g}_{t-1} + \dots \\ &\quad + \beta_2^{t-h-1} \mathbf{g}_{t-h-1} \odot \mathbf{g}_{t-h-1}). \end{aligned}$$

In other words, it is impossible to perform the update by multiple PRNG overwrites of the model weights. As a result, the preconditioner should be maintained in memory before modifying the model weights, which incurs substantial memory cost, as in previous work such as (Zhao et al., 2024b). We aim to remove this additional memory cost in the following subsections.

3.3.1. FINE-SCALED RANDOM STREAM GENERATION BY STATE CACHING

Since saving the entire preconditioner is memory-intensive, we try to perform blockwise preconditioned updates at a

²<https://developer.nvidia.com/curand>

Algorithm 2 PRNG

Input: Seed s
Output: Random number streams r_n
 Initial state mapper I maps the random seed s to the initial state S_0 .
while No stop signal **do**
 PRNG outputs random stream $\{r_n\}$ by the recurrence

$$r_n = O(S_{n-1}), \quad S_n = F(S_{n-1}), \quad (3)$$

end while

lower additional memory cost, which is not available in normal use of PRNG. A formal expression of how concurrent PRNG algorithms generate random number streams is as Algorithm 2, with algorithm-specified and deterministic state update function F and extractor O .

For prior practices including (Malladi et al., 2023; Zhao et al., 2024a;b), the above process guarantees identical and complete z_t for gradient estimation, and gradient updating is obtained by caching the seed s , so that in-place gradient estimation and parameter updating without additional memory access functions. However, caching the random state S offers the possibility to *jump* to a specified position in a random number stream. It allows the PRNG to faithfully continue a particular random stream from wherever it left off, making it more flexible than seed caching. Code examples for this feature can be found in Section C.

An illustration of the block-wise moment approximation is shown in Figure 2. A parameter block partition $w = \{w^{(1)}, w^{(2)}, \dots, w^{(b)}\}$ is prepared at the beginning of a parameter update. We first start at the first block. Processing of the first block is the same as how prior works exploit PRNGs. However, after the first random direction block $z_{t-t'}$ is spawned, AdaMeZO records S_n , the corresponding random state, for each seed. When the spawning of the first block from each of the seeds within the horizon is finished, AdaMeZO skips the initial state mapping in Algorithm 2, and loads the cached S_n to the PRNG for the next block, so that the contiguous random stream is generated rather than starting over again from the first output of the random stream. The process loops until all blocks have finished their updates.

3.3.2. ADAM-STYLE UPDATES WITH ZEROth-ORDER GRADIENTS

With random state caching, we can update models according to Equation (3) block-wise, which is impossible for seed caching as employed by previous works, since seed caching only allows the random stream to be spawned from the initial digit. However, we need some warm-up steps to accumulate history gradients before estimating finite-

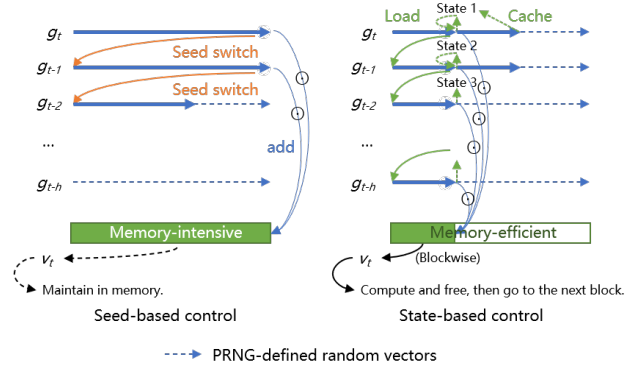


Figure 2. Block-wise moment approximation in AdaMeZO. $\odot$ denotes the Hadamard product.

horizon moments. Due to page limits, we elaborate on the process in Algorithm 3 of the Appendix.

Remark 3.3. It is worth mentioning that caching random states adds a minimal memory cost compared to caching seeds. In Philox (Salmon et al., 2011), the default choice of CUDA PRNG, random state S consists of a 64-bit random seed, a 64-bit subsequence identifier, and a 64-bit offset. The Mersenne Twister (Matsumoto & Nishimura, 1998), the default CPU PRNG, maintains similar information to random states. Therefore, caching the random states incurs a negligible additional memory cost at the bit level compared to caching seeds.

Remark 3.4. Though the first and second moments do not go into the memory, recovering them requires a temporary additional memory trunk, whose size scales to the size of the block, corresponding to the δP term in Table 1. A natural block strategy is the different layers of the model. If the model consists of 32 layers, the additional memory introduced in the second moment approximation is roughly $2/32P$ ($1/32P$ for $m_t^{(b)}$ and $v_t^{(b)}$ each). However, the block can be made as small as a single 1-parameter block. Therefore, AdaMeZO can theoretically approximate the moments by performing frequent random state dumps and loads without incurring additional memory requirements.

4. Theory

We employ the following widely adopted assumptions to facilitate an analysis.

Assumption 4.1 (L-smooth). *For any weight vector $w_1, w_2 \in \mathbb{R}^d$, for a constant $0 < L < \infty$ it holds that*

$$\mathcal{L}(w_2) \leq \mathcal{L}(w_1) + \langle \nabla \mathcal{L}(w_1), w_2 - w_1 \rangle + \frac{L}{2} \|w_2 - w_1\|_2^2.$$

Assumption 4.2 (Bounded gradient variance). *The stochastic gradient $\nabla \mathcal{L}(w_t, \mathcal{B}_t)$ has no bias and σ^2 vari-*

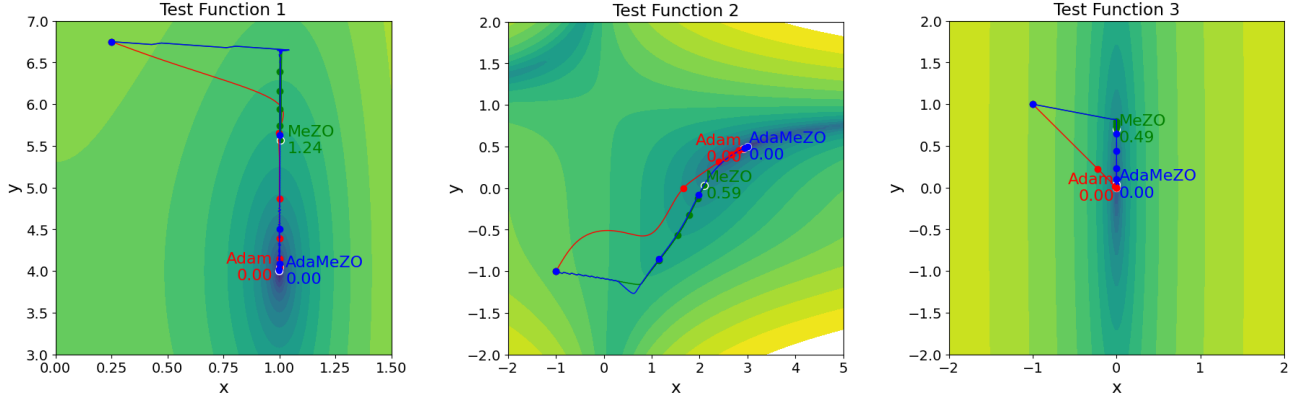


Figure 3. Optimization trajectories on test functions. The loss values at termination are labeled.

ance due to batch stochasticity, specifically

$$\begin{aligned} \mathbb{E}_{\mathcal{B}_t} [\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t)] - \nabla \mathcal{L}(\mathbf{w}_t) &= 0, \\ \mathbb{E}_{\mathcal{B}_t} [\|\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t)\|_2^2] - \|\nabla \mathcal{L}(\mathbf{w}_t)\|_2^2 &\leq \sigma_t^2, \quad \sigma_t < \infty. \end{aligned} \quad (4)$$

Assumption 4.3 (Bounded second moment, (Zhao et al., 2024b)). *Each entry of Σ_t lies in the range $[s_l, s_u]$ with $0 < s_l < s_u < \infty$.*

Assumption 4.4 (Finite gradient drift within horizon). *The gradient drift within the moment horizon h is finite, specifically, $\mathbf{m}_t$ as the first moment at step t satisfies*

$$\|\mathbf{m}_t - \nabla \mathcal{L}(\mathbf{w}_t)\|_2 \leq \mathcal{O}((1 - \beta_1)L\eta).$$

Lemma 4.5 ((Magnus et al., 1978)). *Let A and B be two symmetric matrices, $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$. Define $\mathbf{x} = \mathbf{z}^\top A \mathbf{z} \mathbf{z}^\top B \mathbf{z}$, then it holds that*

$$\mathbb{E}_{\mathbf{z}}[\mathbf{x}] = (\text{tr}A)(\text{tr}B) + 2\text{tr}(AB).$$

Assumption 4.6 (Local r -effective rank, (Malladi et al., 2023)). *Let $G(\mathbf{w}_t) := \max_{\mathcal{B}, |\mathcal{B}|=1} \|\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B})\|$. There is a matrix $\mathcal{H}(\mathbf{w}_t) \preceq LI_d$ satisfying:*

1. *For all $\mathbf{w}$ such that $\|\mathbf{w} - \mathbf{w}_t\|_2 \leq \eta dG(\mathbf{w}_t)$, it holds that $\nabla^2 \mathcal{L}(\mathbf{w}) \preceq H(\mathbf{w}_t)$.*
2. *The effective rank of $H(\mathbf{w}_t)$, specifically, $\text{tr}(\mathcal{H}(\mathbf{w}_t)) / \|\mathcal{H}(\mathbf{w}_t)\|_{\text{op}}$, is at most r .*

We present a convergence bound for non-convex optimization.

Theorem 4.7. *With a sufficiently small learning rate η , AdaMeZO converges to a stationary point with*

$$\mathbb{E} \left[\frac{1}{T} \sum_{t=1}^T \|\nabla \mathcal{L}(\mathbf{w}_t)\|_2^2 \right] \leq \mathcal{O} \left(\frac{1}{\sqrt{T}} \right) + \mathcal{O}(\mu^2).$$

Detailed proof can be found in Appendix E. The bound recovers the structure from (Zhao et al., 2024b). The above result shows that after $T = \mathcal{O}(\epsilon^{-2})$ steps, AdaMeZO converges to a small neighborhood of a stationary point satisfying $\mathbb{E} \left[\left\| \frac{1}{T} \sum_{t=1}^T \nabla \mathcal{L}(\mathbf{w}_t) \right\|_2^2 \right] < \epsilon$.

5. Experiment Results

We present empirical results for AdaMeZO with its baselines in this section. Generally, there are two types of LLMs: 1) encoder-decoder, or masked language models (MLM), such as BERT (Devlin et al., 2019) and its variants, and 2) decoder-only, or autoregressive models (ARM), such as GPT, OPT, and LLaMA families. To comprehensively demonstrate AdaMeZO’s performance, we first illustrate the optimization trajectories for toy functions. Then, we test AdaMeZO with baseline algorithms on well-recognized LLMs, including an MLM RoBERTa (Liu et al., 2019b), and two ARMs, OPT (Zhang et al., 2022a) and LLaMa (Touvron et al., 2023).

5.1. Toy Functions

It is impractical to visualize trajectories of model optimization with billions of dimensions. However, we can illustrate the optimization trajectories on three 2-dimensional toy functions as in Figure 3 to show how AdaMeZO adapts to heterogeneous curvatures. We test the Adam optimizer (Kingma & Ba, 2014) implemented in PyTorch (Paszke, 2019), the vanilla MeZO (Malladi et al., 2023), and the proposed AdaMeZO. More details in Section B.5.

In general, we observe that AdaMeZO shares Adam’s curvature adaptability. Although AdaMeZO walks longer paths due to stochastic gradient directions and warm-up steps, it moves swiftly in regions of low curvature thanks to the preconditioning provided by the diagonal Hessian

Table 2. Results on RoBERTa-large over language tasks with $k = 16$.

Task Type	SST-2 — sentiment —	SST-5	SNLI - natural language inference -	MNLI	RTE	TREC - topic -	Average
Zero-shot	79.0	35.5	50.2	48.8	51.4	32.0	49.4
FO ($\geq 4\times$ memory)	91.8	47.5	77.5	70.0	66.4	85.0	73.0
MeZO	90.6 (1.4)	44.1 (1.0)	67.3 (3.1)	58.1 (1.1)	61.6 (1.3)	67.3 (2.7)	64.8 —
MeZO-switch	90.6 (1.6)	44.3 (1.6)	67.3 (2.8)	58.0 (1.3)	61.6 (2.0)	67.0 (4.7)	64.8 —
AdaMeZO	90.9 (0.9)	45.2 (2.0)	66.8 (2.9)	58.6 (1.4)	63.1 (2.3)	71.5 (5.2)	66.0 —

Table 3. Main results on OPT-1.3B over language tasks. Avg (w.o S,D) indicates the average metric except SQuAD and DROP.

Task Type	SST-2	RTE	CB	BoolQ	WSC	WIC	MultiRC	COPA	ReCoRD	SQuAD	DROP	Avg	Avg (w.o S,D)
	— classification —						- multiple choice -		— generation —				
Zero-shot	53.5	53.4	39.2	45.5	43.2	57.5	45.4	75.0	70.5	27.2	11.1	47.4	53.6
FO ($\geq 4\times$ memory)	90.9 (1.2)	64.0 (10.7)	77.2 (7.9)	64.4 (9.3)	52.8 (2.0)	62.3 (1.9)	65.2 (6.0)	74.0 (2.9)	69.1 (1.2)	80.4 (1.5)	28.2 (1.7)	66.2 —	68.9 —
MeZO	90.9 (0.3)	52.5 (1.5)	65.5 (6.9)	61.8 (2.1)	51.1 (8.4)	58.6 (1.4)	53.7 (2.2)	74.5 (3.6)	70.6 (1.0)	73.3 (0.2)	22.8 (0.6)	61.4 —	64.4 —
MeZO-switch	91.0 (0.6)	53.8 (1.6)	68.7 (2.3)	61.9 (0.6)	52.1 (7.6)	58.3 (1.6)	54.9 (1.5)	75.5 (3.6)	71.0 (1.2)	73.7 (1.2)	24.3 (1.3)	62.3 —	65.2 —
HiZOO	90.9 (1.0)	54.5 (1.6)	63.3 (8.5)	62.7 (1.6)	49.4 (6.9)	58.4 (0.4)	55.4 (1.7)	74.0 (1.8)	70.8 (0.8)	74.5 (0.4)	24.5 (0.5)	61.7 —	64.4 —
AdaMeZO	91.6 (0.3)	54.3 (3.1)	69.6 (1.4)	63.2 (1.6)	53.5 (7.8)	58.4 (1.6)	55.9 (0.7)	75.5 (4.0)	71.1 (1.3)	76.1 (0.7)	24.6 (1.0)	63.1 —	65.9 —

Table 4. Memory profile (MB) on standard PyTorch build, measured on OPT-1.3b, batch size=1. Measured via `nvidia-smi`.

Optimizer	SST2	COPA	SQuAD	Memory
MeZO	5016	5058	5040	1x
Adam	22172	21660	22688	4.40x
HiZOO	7532	7535	7396	1.49x
AdaMeZO	5410	5452	5434	1.07x

estimator, and the final loss values are comparable to those of Adam. In contrast, MeZO struggles with oscillations in low-curvature regions, leading to worse convergence.

5.2. Main Results

We compare the performance of AdaMeZO with vanilla MeZO and MeZO-switch, a variant of MeZO in which the learning rate is manually adjusted to ensure its optimization trajectory is longer than that of AdaMeZO. This ensures that AdaMeZO’s outperformance is not due to MeZO’s underfitting, but rather to its adaptability to the loss landscape. Additionally, we include HiZOO (Zhao et al., 2024b) as a strong baseline that maintains preconditioners in memory. We set $h = 10$, $\beta_1 = 0.7$, $\beta_2 = 0.9$ to evaluate performance across 4 randomly sampled data subsets of the same size and report the mean and standard deviation of the corresponding metric for each task after a hyperparameter study presented in Appendix B.3.

Consistent with previous research (Malladi et al., 2023), we conduct experiments on RoBERTa-large 350M on three types of NLP tasks: sentiment, natural language inference, and topic. We sample $k = 16$ examples per class to demonstrate training performance in a few-shot scenario (see Table 2). It is found that:

AdaMeZO yields better performance. Averaged across all tasks, AdaMeZO achieves a **1.2%** absolute accuracy improvement to MeZO on average, with particularly strong gains in tasks like RTE (**1.5%**), TREC (**4.2%**).

Then we extend our investigation to two autoregressive architectures: OPT (Table 3) and LLaMA3 (Table 5). Experimental results show that:

AdaMeZO’s superior performance scales up to billion-level LLMs. On OPT-1.3B, AdaMeZO surpasses MeZO and MeZO-switch in all but one task. AdaMeZO achieves a **1.7%** absolute accuracy improvement to MeZO on average, with particularly strong gains in tasks like CB (**4.1%**), SQuAD (**2.8%**), WSC (**2.4%**). AdaMeZO also outperforms HiZOO with an average lead of **1.5%**. For LLaMA-3B, AdaMeZO further extends its lead, achieves a **2.7%** absolute accuracy improvement to MeZO on average, with particularly strong gains in tasks like SST2 (**8.1%**), MultiRC (**4.3%**), WSC (**4.1%**), COPA (**4.0%**). AdaMeZO demonstrates its scalability and maintains its advantage in modern-scale models like OPT-30B, as reported in Table 6.

Table 5. Main results on LLaMA3-3B over language tasks.

Task Type	SST-2	RTE	CB	BoolQ	WSC	WIC	MultiRC	COPA	ReCoRD	SQuAD	DROP	Avg	Avg (w.o S,D)
	classification						multiple choice		generation				
Zero-shot	56.0	52.7	51.6	60.9	36.5	54.3	44.8	75.0	68.2	47.3	20.8	51.6	55.5
FO ($\geq 4\times$ memory)	92.5	73.9	85.6	65.9	57.8	67.1	70.6	75.7	68.6	83.9	32.2	70.3	73.1
	(0.7)	(5.4)	(6.9)	(7.3)	(7.6)	(0.7)	(1.8)	(2.6)	(1.0)	(0.3)	(1.8)	—	—
MeZO	84.5	53.2	64.7	62.6	50.4	54.6	52.6	77.2	70.0	79.2	26.8	61.4	63.3
	(4.9)	(0.7)	(2.6)	(0.7)	(11.3)	(0.3)	(2.5)	(2.0)	(0.4)	(0.9)	(0.5)	—	—
MeZO-switch	86.6	54.1	65.5	63.2	51.6	54.7	54.7	78.7	70.4	80.4	27.6	62.5	64.4
	(4.5)	(1.5)	(0.9)	(0.3)	(12.2)	(1.0)	(0.6)	(2.2)	(0.6)	(0.9)	(0.6)	—	—
HiZOO	92.2	54.1	65.1	63.7	52.8	54.9	56.5	82.5	71.5	18.5	6.1	56.1	65.9
	(0.5)	(0.3)	(2.2)	(0.3)	(5.4)	(1.7)	(0.7)	(0.5)	(0.6)	(1.9)	(1.2)	—	—
AdaMeZO	92.6	54.4	66.0	64.6	54.5	54.9	56.9	81.2	71.3	80.4	28.1	64.1	66.3
	(0.5)	(1.5)	(1.4)	(2.6)	(7.5)	(1.6)	(1.0)	(3.2)	(0.9)	(1.8)	(1.1)	—	—

Table 6. Main results on OPT-30B over language tasks, with prefix-tuning.

Task	SST-2	WSC	WIC	COPA	Avg
Zero-shot	56.6	38.4	50.1	81.0	56.5
HiZOO	90.1	56.9	55.2	86.2	72.1
	(1.1)	(6.2)	(3.6)	(1.7)	—
AdaMeZO	91.1	57.6	57.3	87.0	73.2
	(0.6)	(2.4)	(1.5)	(1.4)	—

Table 7. Runtime profile (sec/step) on standard PyTorch build, measured on OPT-1.3b, batch size=1.

Optimizer	SST2	COPA	SQuAD
MeZO	0.21	0.18	0.21
HiZOO	0.23	0.24	0.25
MeZO + a.	0.23	0.23	0.24
MeZO + a. + b. (AdaMeZO)	0.31	0.30	0.31
Adam	0.12	0.13	0.13

We defer results of larger scale to Appendix D.1.

5.3. Memory Efficiency

AdaMeZO incurs a small additional memory due to block-wise moment caching as reported in Table 4. We can observe that, compared to optimizers that maintain actual moments, the additional memory cost is significantly reduced.

5.4. Wall-clock Time Analysis

AdaMeZO incurs longer per-step runtime compared to MeZO, mainly due to a) the additional PRNG calls for past gradient regeneration, and b) the weighted gradient accumulation for moment recovery. We report a runtime profile as Table 7. We observe that the main contributor to AdaMeZO’s additional runtime is the accumulation of past gradients that are regenerated. Optimizing this accumulation process or using prefix tuning will narrow the speed gap relative to MeZO.

6. Conclusion, Limitations and Future Works

In this work, we introduce AdaMeZO, the first ZO optimizer that incorporates Adam-style first- and second-moment updates without doubling or tripling the memory requirements of the original MeZO. This is achieved by estimating truncated moments and performing more refined operations on PRNGs. We provide theoretical analysis and empirical evaluations. Visualizations show that AdaMeZO adapts to complex loss landscapes without consuming excessive additional memory. Experiments on well-recognized models show that AdaMeZO reaches on-par performance using fewer forward passes and can continue to lower loss values before reaching identical terminal conditions. The paper’s limitations are as follows.

We have captured the gradient drift across different steps using a big $\mathcal{O}$ constant related to L-smoothness, EMA weight β_1 , and learning rate. Although finite moment horizons may help to keep the estimations less biased, we did not attempt to explicitly capture the gap, which is a future research direction.

AdaMeZO estimates second moments at a small cost, but they are inaccurate. The reason is two-fold: 1) AdaMeZO runs on zeroth-order gradient estimations, and 2) a smaller β_2 to guarantee that the discarded part contributes only a small share. Future investigations into more accurate second-moment estimations could improve performance.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

Acknowledgments

This work is supported in part by the National Natural Science Foundation of China (Grant No. 62522118,

62371313), in part by the Shenzhen Science and Technology Program (Grant No. JCYJ20241202124934046), in part by the Guangdong Young Talent Research Project (Grant No. 2023TQ07A708), and in part by Shenzhen Loop Area Institute (Contract No. SLAI2026020007).

References

- Agarwal, A., Wainwright, M. J., Bartlett, P., and Ravikumar, P. Information-theoretic lower bounds on the oracle complexity of convex optimization. *Advances in Neural Information Processing Systems*, 22, 2009.
- Beale, E. M. L. *On an iterative method for finding a local minimum of a function of more than one variable*. Number 25. Statistical Techniques Research Group, Section of Mathematical Statistics, Department of Mathematics, Princeton University, 1958.
- Bottou, L., Curtis, F. E., and Nocedal, J. Optimization methods for large-scale machine learning. *SIAM review*, 60(2):223–311, 2018.
- Chen, A., Zhang, Y., Jia, J., Diffenderfer, J., Liu, J., Parasyris, K., Zhang, Y., Zhang, Z., Kailkhura, B., and Liu, S. Deepzero: Scaling up zeroth-order optimization for deep model training. *arXiv preprint arXiv:2310.02025*, 2023.
- Chen, M., Huang, Y.-L., and Wen, Z. Towards efficient low-order hybrid optimizer for language model fine-tuning. 2025a.
- Chen, X., Liang, C., Huang, D., Real, E., Liu, Y., Wang, K., Hsieh, C.-J., Lu, Y., and Le, Q. V. Evolved optimizer for vision. In *First Conference on Automated Machine Learning (Late-Breaking Workshop)*, 2022.
- Chen, Y., Zhang, Y., Cao, L., Yuan, K., and Wen, Z. Enhancing zeroth-order fine-tuning for language models with low-rank structures. *arXiv preprint arXiv:2410.07698*, 2024.
- Chen, Y., Zhang, Y., Liu, Y., Yuan, K., and Wen, Z. A memory efficient randomized subspace optimization method for training large language models. *arXiv preprint arXiv:2502.07222*, 2025b.
- Das, R., Agarwal, N., Sanghavi, S., and Dhillon, I. S. Towards quantifying the preconditioning effect of adam. *arXiv preprint arXiv:2402.07114*, 2024.
- Defazio, A., Yang, X., Mehta, H., Mishchenko, K., Khaled, A., and Cutkosky, A. The road less scheduled. *Advances in Neural Information Processing Systems*, 37: 9974–10007, 2024.
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. Qlora: Efficient finetuning of quantized llms, 2023. URL <https://arxiv.org/abs/2305.14314>, 2, 2023.
- Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pp. 4171–4186, 2019.
- Elsayed, M., Farrahi, H., Dangel, F., and Mahmood, A. R. Revisiting scalable hessian diagonal approximations for applications in reinforcement learning. *arXiv preprint arXiv:2406.03276*, 2024.
- FairScale authors. Fairscale: A general purpose modular pytorch library for high performance and large scale training. <https://github.com/facebookresearch/fairscale>, 2021.
- Ghorbani, B., Krishnan, S., and Xiao, Y. An investigation into neural net optimization via hessian eigenvalue density. In *International Conference on Machine Learning*, pp. 2232–2241. PMLR, 2019.
- Gui, M., Zhao, Z., Qiu, T., and Shen, H. Laplace ap-approximation with diagonalized hessian for over-parameterized neural networks. In *NeurIPS Workshop on Bayesian Deep Learning*, 2021.
- Guo, W., Long, J., Zeng, Y., Liu, Z., Yang, X., Ran, Y., Gardner, J. R., Bastani, O., De Sa, C., Yu, X., et al. Zeroth-order fine-tuning of llms with extreme sparsity. *arXiv preprint arXiv:2406.02913*, 2024.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.
- Jamieson, K. G., Nowak, R., and Recht, B. Query complexity of derivative-free optimization. *Advances in Neural Information Processing Systems*, 25, 2012.
- Jiang, S., Chen, Q., Pan, Y., Xiang, Y., Lin, Y., Wu, X., Liu, C., and Song, X. Zo-adamu optimizer: Adapting perturbation by the momentum and uncertainty in zeroth-order optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 18363–18371, 2024.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Kunstner, F., Chen, J., Lavington, J. W., and Schmidt, M. Noise is not the main factor behind the gap between sgd and adam on transformers, but sign descent might be. *arXiv preprint arXiv:2304.13960*, 2023.

- Lester, B., Al-Rfou, R., and Constant, N. The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*, 2021.
- Li, X. L. and Liang, P. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021.
- Liu, L., Jiang, H., He, P., Chen, W., Liu, X., Gao, J., and Han, J. On the variance of the adaptive learning rate and beyond. *arXiv preprint arXiv:1908.03265*, 2019a.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019b.
- Liu, Y., Zhu, Z., Gong, C., Cheng, M., Hsieh, C.-J., and You, Y. Sparse mezo: Less parameters for better performance in zeroth-order llm fine-tuning. *arXiv preprint arXiv:2402.15751*, 2024.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- Luo, L., Xiong, Y., Liu, Y., and Sun, X. Adaptive gradient methods with dynamic bound of learning rate. *arXiv preprint arXiv:1902.09843*, 2019.
- Magnus, J. R. et al. *The moments of products of quadratic forms in normal variables*. Univ., Instituut voor Actuarie en Econometrie, 1978.
- Malladi, S., Gao, T., Nichani, E., Damian, A., Lee, J. D., Chen, D., and Arora, S. Fine-tuning language models with just forward passes. *Advances in Neural Information Processing Systems*, 36:53038–53075, 2023.
- Matsumoto, M. and Nishimura, T. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, 8(1):3–30, 1998.
- Pan, R., Liu, X., Diao, S., Pi, R., Zhang, J., Han, C., and Zhang, T. Lisa: layerwise importance sampling for memory-efficient large language model fine-tuning. *Advances in Neural Information Processing Systems*, 37: 57018–57049, 2024.
- Paszke, A. Pytorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703*, 2019.
- Pethick, T., Xie, W., Antonakopoulos, K., Zhu, Z., Silveti-Falls, A., and Cevher, V. Training deep learning models with norm-constrained lmos. *arXiv preprint arXiv:2502.07529*, 2025.
- Raginsky, M. and Rakhlin, A. Information-based complexity, feedback and dynamics in convex programming. *IEEE Transactions on Information Theory*, 57(10):7036–7056, 2011.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.
- Sagun, L., Bottou, L., and LeCun, Y. Eigenvalues of the hessian in deep learning: Singularity and beyond. *arXiv preprint arXiv:1611.07476*, 2016.
- Salmon, J. K., Moraes, M. A., Dror, R. O., and Shaw, D. E. Parallel random numbers: as easy as 1, 2, 3. In *Proceedings of 2011 international conference for high performance computing, networking, storage and analysis*, pp. 1–12, 2011.
- Shazeer, N. and Stern, M. Adafactor: Adaptive learning rates with sublinear memory cost. In *International Conference on Machine Learning*, pp. 4596–4604. PMLR, 2018.
- Spall, J. C. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE transactions on automatic control*, 37(3):332–341, 1992.
- Spall, J. C. A one-measurement form of simultaneous perturbation stochastic approximation. *Automatica*, 33(1): 109–112, 1997.
- Sun, Y., Huang, T., Ding, L., Shen, L., and Tao, D. Tezo: Empowering the low-rankness on the temporal dimension in the zeroth-order optimization for fine-tuning llms. *arXiv preprint arXiv:2501.19057*, 2025.
- Tan, Q., Liu, J., Zhan, Z., Ding, C., Wang, Y., Lu, J., and Yuan, G. Harmony in divergence: Towards fast, accurate, and memory-efficient zeroth-order llm fine-tuning. *arXiv preprint arXiv:2502.03304*, 2025.
- Tang, Z., Rybin, D., and Chang, T.-H. Zeroth-order optimization meets human feedback: Provable learning via ranking oracles. *arXiv preprint arXiv:2303.03751*, 2023.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Vakhitov, A. T., Granichin, O. N., and Gurevich, L. S. Algorithm for stochastic approximation with trial input perturbation in the nonstationary problem of optimization. *Automation and Remote Control*, 70:1827–1835, 2009.

- Wang, Z., Balasubramanian, K., Ma, S., and Razaviyayn, M. Zeroth-order algorithms for nonconvex minimax problems with improved complexities. *arXiv preprint arXiv:2001.07819*, 2020.
- You, Y., Li, J., Reddi, S., Hseu, J., Kumar, S., Bhojanapalli, S., Song, X., Demmel, J., Keutzer, K., and Hsieh, C.-J. Large batch optimization for deep learning: Training bert in 76 minutes. *arXiv preprint arXiv:1904.00962*, 2019.
- Yu, Z., Cheng, Y., Ding, L., Tian, X., Shen, L., and Tao, D. Memory-efficient block coordinate descent for hessian-informed zeroth-order optimizer.
- Zhang, L., Shi, S., and Li, B. Eva: Practical second-order optimization with kronecker-vectorized approximation. In *The Eleventh International Conference on Learning Representations*, 2023.
- Zhang, S., Roller, S., Goyal, N., Artetxe, M., Chen, M., Chen, S., Dewan, C., Diab, M., Li, X., Lin, X. V., et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022a.
- Zhang, Y., Chen, C., Shi, N., Sun, R., and Luo, Z.-Q. Adam can converge without any modification on update rules. *Advances in neural information processing systems*, 35: 28386–28399, 2022b.
- Zhang, Y., Chen, C., Ding, T., Li, Z., Sun, R., and Luo, Z. Why transformers need adam: A hessian perspective. *Advances in Neural Information Processing Systems*, 37: 131786–131823, 2024a.
- Zhang, Y., Chen, C., Li, Z., Ding, T., Wu, C., Kingma, D. P., Ye, Y., Luo, Z.-Q., and Sun, R. Adam-mini: Use fewer learning rates to gain more. *arXiv preprint arXiv:2406.16793*, 2024b.
- Zhang, Y., Li, P., Hong, J., Li, J., Zhang, Y., Zheng, W., Chen, P.-Y., Lee, J. D., Yin, W., Hong, M., et al. Revisiting zeroth-order optimization for memory-efficient llm fine-tuning: A benchmark. *arXiv preprint arXiv:2402.11592*, 2024c.
- Zhao, H., Li, J., Pan, Y., Liang, S., Yang, X., Liu, W., Li, X., Dou, F., Liu, T., and Lu, J. Helene: Hessian layer-wise clipping and gradient annealing for accelerating fine-tuning llm with zeroth-order optimization. *arXiv preprint arXiv:2411.10696*, 2024a.
- Zhao, Y., Dang, S., Ye, H., Dai, G., Qian, Y., and Tsang, I. W. Second-order fine-tuning without pain for llms: A hessian informed zeroth-order optimizer. *arXiv preprint arXiv:2402.15173*, 2024b.
- Zhuang, J., Tang, T., Ding, Y., Tatikonda, S. C., Dvornek, N., Papademetris, X., and Duncan, J. Adabelief optimizer: Adapting stepsizes by the belief in observed gradients. *Advances in neural information processing systems*, 33: 18795–18806, 2020.

A. Additional Related Works

In addition to MeZO, numerous subsequent excellent works have emerged to enhance the vanilla version. (Jiang et al., 2024) incorporates uncertain moments estimations to promote convergence. (Zhao et al., 2024a) invokes Adam-style update rules for better performance. (Zhao et al., 2024b) estimates the diagonal Hessian with a three-point second derivative estimation, admitting a third forward pass for each step. (Liu et al., 2024; Guo et al., 2024) proposed to insert sparsity for better performance. (Chen et al., 2024; Sun et al., 2025) exploits the low-rank property for better performance. (Chen et al., 2025a) proposes a hybrid optimizer that balances efficiency and trade-offs. (Tan et al., 2025) explores a layer-wise adaptation to speed up zeroth-order fine-tuning. (Chen et al., 2025b) investigates memory-efficient zeroth-order fine-tuning from a subspace-optimization perspective. (Yu et al.) introduces a block version of HiZOO, attempting to preserve preconditioning-improved convergence while reducing additional memory access.

B. Detailed Experiment Settings

B.1. Computation Resources

We summarize the computational devices for empirical evaluations in Table 8. We use device 1 for MLM experiments and device 2 for ARM experiments.

Table 8. Summary of computational devices for empirical evaluations.

Device	OS/CPU/GPU	Python	PyTorch	CUDA	cuDNN
1	Linux 5.10.0, amd64				
	Intel(R) Xeon(R) Gold 6133 CPU @ 2.50GHz 6x NVIDIA GeForce RTX 3090 GPU	3.10.13	2.3.0	12.1	8.9
2	Linux 4.18.0, x86_64				
	AMD EPYC 7742 64-Core Processor 4x NVIDIA A100-SXM4-80GB	3.9.7	2.1.0	12.1	8.9

B.2. Formal Pseudo-codes for AdaMeZO

A formal description of AdaMeZO in pseudo-code is as Algorithm 3.

B.3. Hyperparameter Study

There are 3 new hyperparameters introduced in AdaMeZO, the horizon h , and the EMA weight in momentum update (β_1, β_2) as in Adam.

We first analyze how different h impacts the evaluation loss curve, as shown in Figure 4. We find that setting $h > 5$ admits comparable results, while $h < 5$ requires further optimization to avoid crash due to numerical instability. We set $h = 10$ in experiments for stability, and set $h = 5$ for wall-clock time analysis.

Similar to Adam, two new hyperparameters (β_1, β_2) are introduced into the algorithm. We report AdaMeZO’s performance across different hyperparameter settings, as shown in Table 9, on OPT-1.3B with the SST2 task, since complete task sweeps are computationally expensive. It can be observed that the first moments improve MeZO’s performance, and the second moments further improve it. The performance gain is robust against reasonable choices of the hyperparameter (β_1, β_2) .

Table 9. Performance comparison with different (β_1, β_2) .

(β_1, β_2)	SST2	COPA	SQuAD
(0.7, 0.9)	91.6 (0.3)	75.5 (4.0)	76.1 (0.7)
(0.7, 0.99)	90.9 (0.9)	75.3 (2.9)	75.6 (0.9)
(0.6, 0.9)	91.1 (0.6)	75.8 (2.9)	75.6 (1.2)
(0.8, 0.9)	91.5 (0.7)	74.3 (2.9)	75.6 (1.5)
(0.7, 0.0), mSGD	90.9 (0.9)	75.3 (2.9)	75.6 (0.9)
(0.0, 0.0), MeZO	90.9 (0.3)	74.5 (3.6)	73.3 (0.2)

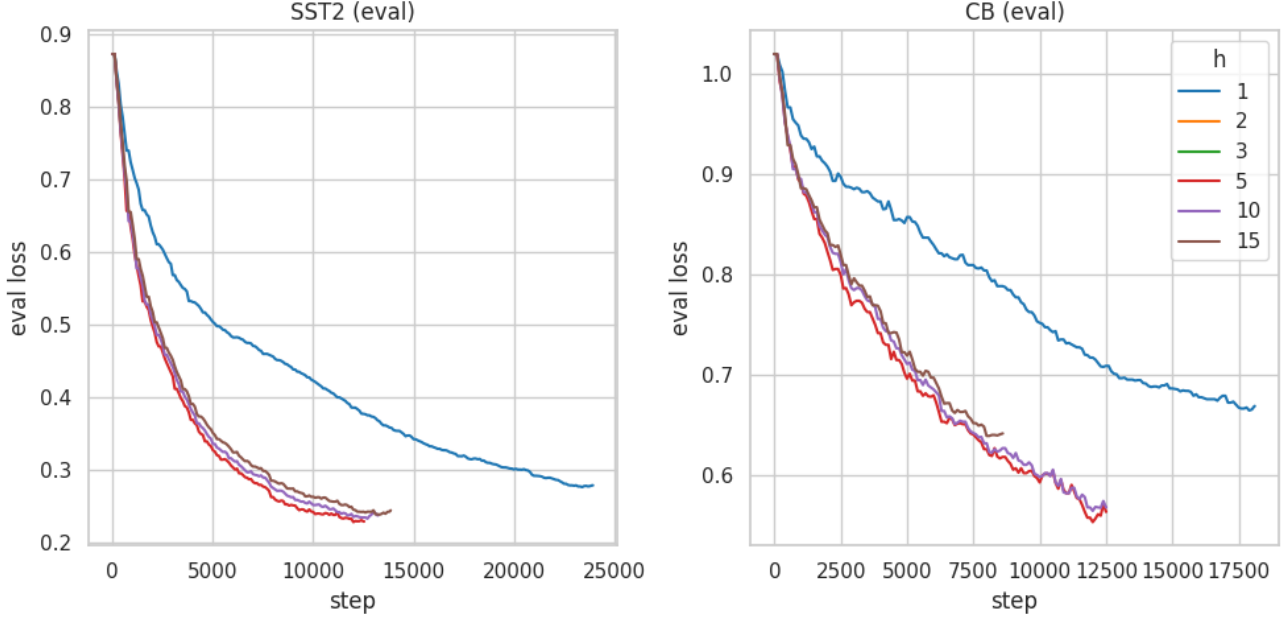


Figure 4. Evaluation loss with different h .

B.4. Detailed Settings for Figure 1

For (a), the learning rate is $1e-6$, with 16 training samples per class. For (b) and (c), the learning rate is $1e-7$, with 1000 training samples in total. We set $\beta_1 = 0.7, \beta_2 = 0.9, h = 10$, so that AdaMeZO discards only a small part by truncating the moments and admits a smoother second moment estimation compared to the first. With an abuse of context, the choice of (β_1, β_2) falls into the suggested area $0 < \beta_1 < \sqrt{\beta_2} < 1$ by (Zhang et al., 2022b). Fine-tuning terminates when either of the following happens.

1. Measure evaluation loss per 100 steps. Evaluation loss does not drop for 5 continual measures.
2. Number of steps exceeds 40000.

B.5. Detailed Settings for Section 5.1

The expressions of the test functions are

1. $f_1(x, y) = 8(x - 1)^2(1.3x^2 + 2x + 1) + 0.5(y - 4)^2$ (Zhao et al., 2024b).
2. $f_2(x, y) = (1.5 - x + xy)^2 + (2.25 - x + xy^2)^2 + (2.625 - x + xy^3)^2$ (Beale, 1958).
3. $f_3(x, y) = 100x^2 + y^2$.

Specifications on implementations are as Table 10. The setting follows on the following rule:

1. Set the learning rate of Adam to 0.01,
2. Tune the learning rate for ZO optimizers so that the trajectory lengths are comparable to Adam's. We allow a longer trajectory ($< 1.6\times$) for ZO optimizers.

For MeZO and AdaMeZO, we allow only 2 seeds coding 2 gradient directions. This is to capture the situation where the number of steps, equivalently the total number of explored gradient directions (in thousands), is usually less than the number of dimensions of the LLMs (in billions).

Trajectories at higher resolutions and 3D views of the loss landscapes are shown in Figure 5.

Table 10. Specifications for toy functions.

	Adam		MeZO		AdaMeZO		# steps	Initialization
	lr	length	lr	length	lr	length		
f_1	0.01	3.0227	0.01	4.6659	0.01	4.5078	600	(0.2, 6.75)
f_2	0.01	4.3597	0.002	5.5405	0.002	5.3207	2500	(-1, -1)
f_3	0.01	1.4142	0.01	1.4243	0.01	1.8577	500	(-1, 1)

B.6. Detailed Settings for Section 5.2

Table 11. Hyperparameter settings.

	B	T	η	q	μ	(β_1, β_2)
Table 2	16	1×10^5	1×10^{-6}	5	1×10^{-3}	(0.7, 0.9)
Table 3	16	4×10^4	1×10^{-7}	5	1×10^{-3}	(0.7, 0.9)
Table 5	16	4×10^4	1×10^{-7}	5	1×10^{-3}	(0.7, 0.9)

Fine-tuning terminates when either of the following conditions is met.

1. Measure evaluation loss per 100 steps. Evaluation loss does not drop for q continual measures.
2. Number of steps exceeds T .

C. Code Snippets for Block-wise Gradient Generation

Previous works call PRNG by the following codes.

```
torch.manual_seed(seed)
z = torch.normal(
    mean=0,
    std=1,
    size=param.data.size(),
    dtype=param.data.dtype,
    device=param.device,
)
```

In this work, for block-wise gradient generation, we wish the PRNG to skip the random stream belonging to prior blocks. Therefore, we directly feed random states into the PRNG, thereby skipping the initialization step implied by `manual_seed(seed)`. A snippet to realize the feature is as follows.

```
self.g = torch.Generator(device='cuda')
self.g.set_state(state) # scheduled state
z = torch.normal(
    mean=0,
    std=1,
    size=param.data.size(),
    dtype=param.data.dtype,
    device=param.device,
    generator=self.g,
)
state = self.g.get_state(state)
```

D. Additional Experiment Results

D.1. Larger Models

We report the performance of AdaMeZO on larger models to demonstrate the scalability of the optimizer as Table 12 and Table 13.

We also report the hyperparameter settings in Section D.1 for Table 6.

We include training loss and evaluation loss curves in Figure 6 and Figure 7, respectively.

Table 12. Main results on LLaMA-7B over language tasks.

Task Type	SST-2	RTE	CB	BoolQ	WSC	WIC	MultiRC	COPA	ReCoRD	SQuAD	DROP	Avg	Avg (w.o S,D)
	classification							multiple choice		generation			
Zero-shot	59.7	49.8	48.2	65.0	56.7	50.6	50.5	84.0	79.9	58.6	17.5	56.4	60.4
FO ($\geq 4\times$ memory)	95.0	86.0	94.1	83.1	54.5	66.2	79.3	81.2	75.4	89.2	39.7	76.7	79.4
	(0.5)	(2.2)	(1.7)	(0.5)	(5.8)	(4.9)	(3.0)	(2.2)	(2.3)	(1.0)	(1.0)	—	—
MeZO	85.7	54.7	58.8	68.3	58.1	56.9	60.9	82.5	78.0	71.9	30.9	64.2	67.1
	(1.9)	(0.5)	(3.8)	(1.5)	(2.9)	(1.7)	(2.7)	(1.2)	(1.8)	(4.5)	(1.1)	—	—
MeZO-switch	87.2	55.2	60.6	68.7	60.2	56.8	60.5	84.0	80.3	78.8	32.3	65.8	68.1
	(0.7)	(1.2)	(6.3)	(1.2)	(1.2)	(0.5)	(2.3)	(0.8)	(0.5)	(3.2)	(1.1)	—	—
HiZOO	90.9	59.7	63.3	70.3	59.8	57.4	62.7	83.7	79.3	21.3	4.6	59.4	69.7
	(2.5)	(2.9)	(0.9)	(1.5)	(6.7)	(0.2)	(2.4)	(1.2)	(1.6)	(1.1)	(0.9)	—	—
AdaMeZO	91.4	61.2	62.9	70.9	60.5	57.6	62.1	84.5	80.5	84.9	36.2	68.4	70.2
	(2.5)	(2.6)	(1.6)	(2.2)	(2.0)	(1.1)	(2.6)	(3.1)	(0.9)	(0.9)	(2.1)	—	—

Table 13. Main results on OPT-13B over language tasks. OOM indicates that HiZOO encountered an out-of-memory error. A cell is marked as OOM if any of the evaluation seeds trigger an OOM. The official HiZOO implementation only supports single-GPU training, and the memory overhead of some instances exceeds the capacity of our largest GPU (A100 80GB), leading to OOM failures.

Task Type	SST-2	RTE	CB	BoolQ	WSC	WIC	MultiRC	COPA	ReCoRD	SQuAD	DROP	Avg	Avg (w.o S,D)
	classification							multiple choice		generation			
Zero-shot	58.8	59.6	46.4	59.0	38.5	55.0	46.9	80.0	81.2	46.2	14.6	53.2	58.3
FO ($\geq 4\times$ memory)	92.0	70.8	83.9	77.1	63.5	55.0	71.1	79.0	74.1	84.9	31.3	71.1	74.0
MeZO	92.1	60.4	67.8	65.5	56.6	54.9	56.7	87.0	80.2	82.1	30.6	66.7	69.0
	(0.5)	(0.6)	(1.4)	(3.0)	(7.9)	(1.7)	(0.8)	(1.1)	(1.0)	(1.3)	(1.5)	—	—
MeZO-switch	92.6	61.6	66.9	66.2	56.9	55.4	57.5	86.0	80.5	83.4	30.5	67.0	69.3
	(0.2)	(2.5)	(1.0)	(3.7)	(8.4)	(0.7)	(0.5)	(2.7)	(1.1)	(0.8)	(0.9)	—	—
HiZOO	91.5	62.5	68.2	OOM	56.4	55.4	57.5	86.2	80.0	83.5	OOM	—	—
	(1.3)	(3.7)	(2.2)	—	(8.3)	(1.4)	(0.6)	(0.9)	(1.4)	(1.2)	—	—	—
AdaMeZO	92.7	63.0	67.8	70.6	58.4	55.8	58.3	87.0	80.1	83.7	31.0	68.0	70.4
	(0.5)	(6.1)	(2.5)	(3.6)	(7.5)	(0.6)	(0.3)	(1.1)	(0.7)	(1.3)	(0.9)	—	—

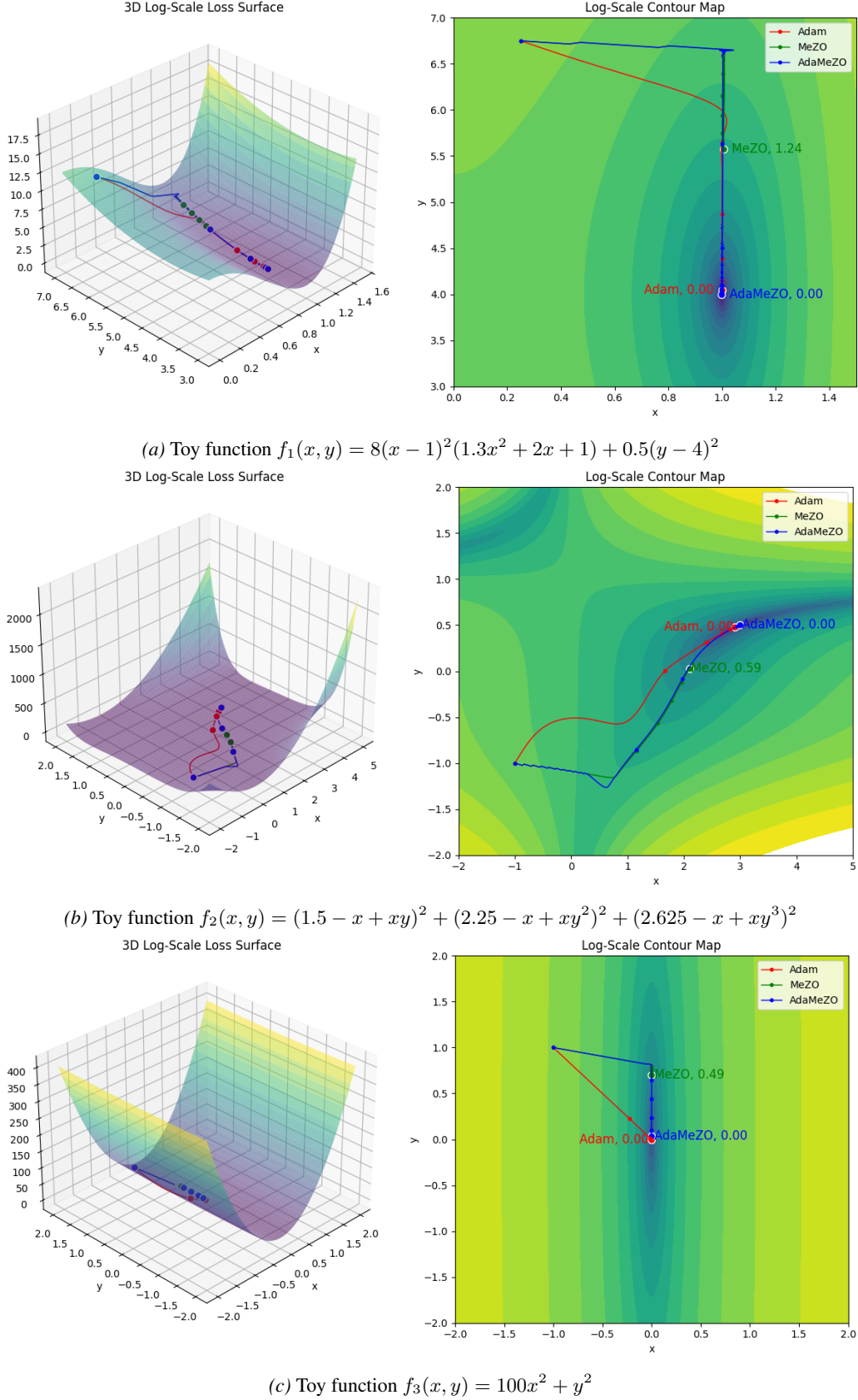


Figure 5. Loss landscapes of the toy functions and optimization trajectories.

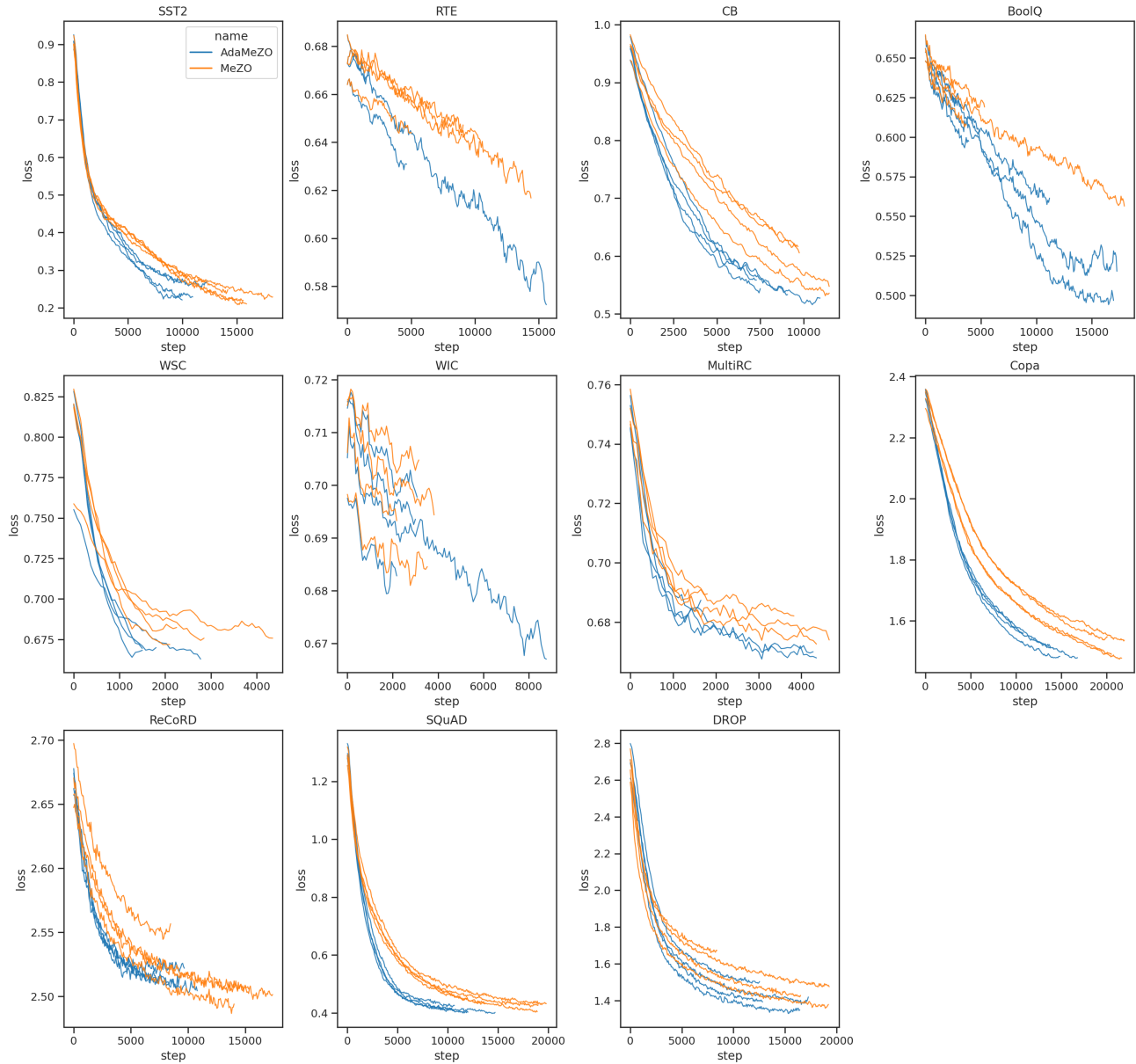


Figure 6. Training loss curve of OPT-13B over language tasks.

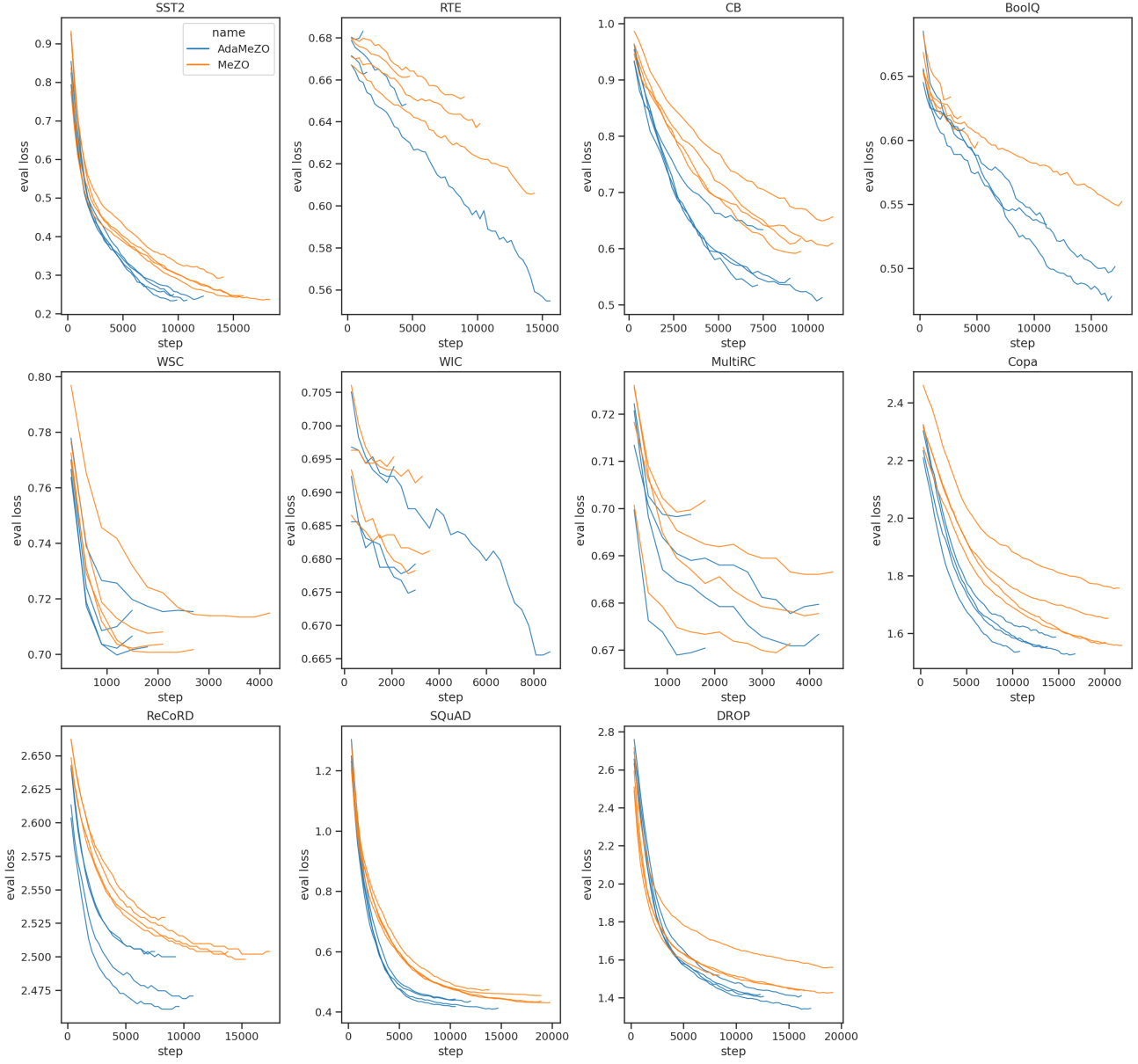


Figure 7. Evaluation loss curve of OPT-13B over language tasks.

Table 14. Hyperparameter settings for Table 6.

Experiment	Hyperparameters	Values
HiZOO (prefix)	B	16
	η	$\{5e-2, 1e-2, 5e-3\}$
	μ	$1e-1$
	# prefix tokens	5
AdaMeZO (prefix)	B	16
	η	$\{7.5e-6, 1e-5, 2.5e-5\}$
	μ	$1e-1$
	# prefix tokens	5

Table 15. Hyperparameter settings for HiZOO in Table 3, Table 5, Table 12, and Table 13.

Experiment	Hyperparameters	Values
HiZOO	B	16
	η	$\{1e-6, 5e-7, 1e-7\}$
	μ	$1e-3$

Algorithm 3 AdaMeZO

Input: Initialized model parameters $\mathbf{w}_0 \in \mathbb{R}^d$, loss function $\mathcal{L} : \mathbb{R}^d \rightarrow \mathbb{R}$, step budget T , perturbation scale μ , learning rate η , horizon h , first EMA ratio β_1 , second EMA ratio β_2 , block strategy $B(\mathbf{w}) = \{\mathbf{w}^{(1)}, \dots, \mathbf{w}^{(b)}\}$, cancel factor β_v , warm-up steps T_w

Output: Trained model parameters $\mathbf{w}_T$

seeds, projs $\leftarrow []$, $[]$

for $t = 1, \dots, T$ **do**

 Sample batch $\mathcal{B}_t$ and random seed s

 Reset the PRNG with random seed s , spawn $\mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, I_d)$

 Estimate p_t using Equation (1)

 seeds.append(s), projs.append(p_t)

in-place model perturbation

$\mathbf{w}_t \leftarrow \mathbf{w}_t$

if $t > T_w$ **then**

 states $\leftarrow [\text{None}] * (h, b)$

for $\tau_b = 1, \dots, b$ **do**

$\mathbf{m}, \mathbf{v} \leftarrow \mathbf{0}, \mathbf{0}$

for $\tau_h = 1, \dots, h$ **do**

$p \leftarrow \text{projs}[-\tau_h]$

if states[τ_h, τ_b] == None **then**

$s \leftarrow \text{seeds}[-\tau_h]$

 Reset the PRNG with random seed s , spawn $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I_{|\mathbf{w}^{(\tau_b)}|})$

else

 Load states[τ_h, τ_b] to PRNG, spawn $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I_{|\mathbf{w}^{(\tau_b)}|})$

end if

 Save PRNG state to states[τ_h, τ_b]

$\mathbf{m} \leftarrow \mathbf{m} + \beta_1^{\tau_h-1} p \mathbf{z}$

$\mathbf{v} \leftarrow \mathbf{v} + \beta_2^{\tau_h-1} p^2 (\mathbf{z} \odot \mathbf{z})$

end for

end for

$\mathbf{w}_t^{(\tau_b)} \leftarrow \mathbf{w}_t^{(\tau_b)} - \eta \beta_v \frac{\mathbf{m}}{\sqrt{\mathbf{v} + \epsilon}}$

else

 Reset the PRNG with random seed s , spawn $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, I_d)$

$\mathbf{w}_t \leftarrow \mathbf{w}_t - \eta p_t \mathbf{z}$

end if

end for

E. Detailed Convergence Analysis

Lemma E.1 (Update expectations). *Given Assumption 4.2 to 4.4 and Assumption 4.5, for warm-up steps, it holds that*

$$\mathbb{E}[\mathbf{u}_t] = \nabla \mathcal{L}(\mathbf{w}_t) + \mathcal{O}(\mu), \quad (5)$$

$$\mathbb{E}[\|\mathbf{u}_t\|_2^2] \leq \frac{\eta^2 L \mathcal{O}(r)}{2} (\|\nabla \mathcal{L}(\mathbf{w}_t)\|_2^2 + \sigma^2) + \mathcal{O}(\mu^2). \quad (6)$$

After warm-up steps, it holds that

$$\mathbb{E}[\mathbf{u}_t] = \Sigma_t^{-1} (\nabla \mathcal{L}(\mathbf{w}_t) + \mathcal{O}(\bar{\beta}_1 L \eta)) + \mathcal{O}(\mu), \quad (7)$$

$$\mathbb{E}[\|\mathbf{u}_t\|_2^2] \leq (2\text{tr}(\Sigma_t^{-1}) + 4s_t^{-1}) (\|\nabla \mathcal{L}(\mathbf{w}_t)\|_{\Sigma_t^{-1}}^2 + \sigma^2 + \mathcal{O}(\bar{\beta}_1^2 L^2 \eta^2)) + \mathcal{O}(\mu^2), \quad (8)$$

where σ captures the batch stochasticity in first-order, $\bar{\beta}_1 = 1 - \beta_1$, and Σ_t is the diagonal matrix with $\mathbf{v}_t$ being its diagonal.

Proof. The bounds for the warm-up phase follow Proof of Lemma 2 in (Malladi et al., 2023).

After the warm-up case, by the definition of $\mathbf{u}_t$, we have

$$\mathbf{u}_t = \Sigma_t^{-1} \mathbf{m}_t,$$

where

$$\Sigma_t := \beta_v \sqrt{\sum_{i=0}^{h-1} \beta_2^i \text{diag} \left(\frac{1}{n} \sum_{j=1}^n \mathbf{g}_{t-i,j} \odot \mathbf{g}_{t-i,j} \right)}, \quad \mathbf{g}_{t,j} := \mathbf{z}_j^\top \nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) \mathbf{z}_j,$$

with β_v is a normalizing factor connected to β_1 and β_2 to cancel out all β_1 and β_2 related terms.

Moreover,

$$\begin{aligned} & \mathbb{E} [\|\mathbf{u}_t\|_2^2] \\ &= \mathbb{E}_{\mathcal{B}_t, \mathbf{z}_j} \left[\left\| \frac{1}{n} \sum_{j=1}^n \Sigma_t^{-\frac{1}{2}} \mathbf{z}_j \mathbf{z}_j^\top \Sigma_t^{-\frac{1}{2}} (\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) + \mathcal{O}(\bar{\beta}_1 L \eta)) + \mathcal{O}(\mu) \right\|_2^2 \right] \\ &\stackrel{(a)}{\leq} 2 \mathbb{E}_{\mathcal{B}_t, \mathbf{z}} \left[\left\| \frac{1}{n} \sum_{j=1}^n \Sigma_t^{-\frac{1}{2}} \mathbf{z}_j \mathbf{z}_j^\top \Sigma_t^{-\frac{1}{2}} (\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) + \mathcal{O}(\bar{\beta}_1 L \eta)) \right\|_2^2 \right] + \mathcal{O}(\mu^2) \\ &\stackrel{(b)}{\leq} \frac{2}{n} \sum_{j=1}^n \mathbb{E}_{\mathcal{B}_t, \mathbf{z}_j} \left[\left\| \Sigma_t^{-\frac{1}{2}} \mathbf{z}_j \mathbf{z}_j^\top \Sigma_t^{-\frac{1}{2}} (\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) + \mathcal{O}(\bar{\beta}_1 L \eta)) \right\|_2^2 \right] + \mathcal{O}(\mu^2) \\ &\stackrel{(c)}{=} 2\text{tr}(\Sigma_t^{-1}) \mathbb{E}_{\mathcal{B}_j} [(\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) + \mathcal{O}(\bar{\beta}_1 L \eta))^\top \Sigma_t^{-1} (\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) + \mathcal{O}(\bar{\beta}_1 L \eta))] \\ &\quad + 4 \mathbb{E}_{\mathcal{B}_j} [(\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) + \mathcal{O}(\bar{\beta}_1 L \eta))^\top \Sigma_t^{-2} (\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) + \mathcal{O}(\bar{\beta}_1 L \eta))] + \mathcal{O}(\mu^2) \\ &\stackrel{(d)}{\leq} (2\text{tr}(\Sigma_t^{-1}) + 4s_t^{-1}) \mathbb{E}_{\mathcal{B}_j} [(\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) + \mathcal{O}(\bar{\beta}_1 L \eta))^\top \Sigma_t^{-1} (\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) + \mathcal{O}(\bar{\beta}_1 L \eta))] + \mathcal{O}(\mu^2) \\ &\stackrel{(e)}{\leq} (2\text{tr}(\Sigma_t^{-1}) + 4s_t^{-1}) (\|\nabla \mathcal{L}(\mathbf{w}_t)\|_{\Sigma_t^{-1}}^2 + \mathcal{O}(\bar{\beta}_1 L \eta) + s_t^{-1} \sigma^2) + \mathcal{O}(\bar{\beta}_1 L \eta). \end{aligned}$$

where (a) is by $\|a + b\|_2^2 \leq \|a\|_2^2 + \|b\|_2^2 + 2\|ab\|_2 \leq 2\|a\|_2^2 + 2\|b\|_2^2$; (b) is by the convexity of the function $\|\cdot\|_2^2$; (c) is by setting $A = \mathbb{E}_{\mathcal{B}_j} [\Sigma_t^{-\frac{1}{2}} (\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) + \mathcal{O}(\bar{\beta}_1 L \eta))^\top (\nabla \mathcal{L}(\mathbf{w}_t, \mathcal{B}_t) + \mathcal{O}(\bar{\beta}_1 L \eta)) \Sigma_t^{-\frac{1}{2}}]$ and $B = \Sigma_t^{-1}$, then apply Assumption 4.5; (d) is by Assumption 4.3; finally (e) by Assumption 4.2. $\square$

Finally, we establish Theorem 4.7.

Proof. Split the full summation into the warm-up phase and the post-warm-up phase as follows.

$$\frac{1}{T} \sum_{t=1}^T \|\nabla \mathcal{L}(\mathbf{w}_t)\|_2^2 = \underbrace{\frac{1}{T} \sum_{t=1}^{T_w} \|\nabla \mathcal{L}(\mathbf{w}_t)\|_2^2}_{\text{warm-up}} + \frac{1}{T} \sum_{t=T_w+1}^T \|\nabla \mathcal{L}(\mathbf{w}_t)\|_2^2.$$

Choose

$$\eta \leq \min \left\{ \frac{1}{s(\text{tr}\Sigma_t^{-1} + 2s_l^{-1})\sqrt{T}}, \frac{1}{L\mathcal{O}(r)\sqrt{T}}, \frac{1}{s\mathbb{E}[\mathcal{L}(\mathbf{w}_1)] - \mathbb{E}[\mathcal{L}(\mathbf{w}_T)]\sqrt{T}} \right\},$$

Equation (5) and (6) with Assumption 4.1 yields

$$\begin{aligned} \mathbb{E}[\mathcal{L}(\mathbf{w}_{t+1})] &\leq \mathcal{L}(\mathbf{w}_t) - \eta \|\nabla \mathcal{L}(\mathbf{w}_t)\|_2^2 + \frac{\eta^2 L \mathcal{O}(r)}{2} (\|\nabla \mathcal{L}(\mathbf{w}_t)\|_2^2 + \sigma^2) + \mathcal{O}(\mu^2) \\ &\leq \mathcal{L}(\mathbf{w}_t) - \frac{\eta}{2} \|\nabla \mathcal{L}(\mathbf{w}_t)\|_2^2 + \frac{\eta^2 L \sigma^2 \mathcal{O}(r)}{2} + \mathcal{O}(\mu^2). \end{aligned}$$

Equation (7) and (8) with Assumption 4.1 yields

$$\begin{aligned} \mathbb{E}[\mathcal{L}(\mathbf{w}_{t+1})] &\leq \mathcal{L}(\mathbf{w}_t) - \eta \|\nabla \mathcal{L}(\mathbf{w}_t)\|_{\Sigma_t^{-1}}^2 + L\eta^2 (\text{tr}\Sigma_t^{-1} + 2s_l^{-1}) \left(\|\nabla \mathcal{L}(\mathbf{w}_t)\|_{\Sigma_t^{-1}}^2 + \mathcal{O}(\bar{\beta}_1^2 L^2 \eta^2) + \sigma^2 \right) + \mathcal{O}(\mu^2) \\ &\leq \mathcal{L}(\mathbf{w}_t) - \frac{\eta}{2} \|\nabla \mathcal{L}(\mathbf{w}_t)\|_{\Sigma_t^{-1}}^2 + L\eta^2 (\mathcal{O}(\bar{\beta}_1^2 L^2 \eta^2) + \sigma^2) (\text{tr}\Sigma_t^{-1} + 2s_l^{-1}) + \mathcal{O}(\mu^2). \end{aligned}$$

So, for the warm-up phase,

$$\frac{1}{T} \sum_{t=1}^{T_w} \|\nabla \mathcal{L}(\mathbf{w}_t)\|_2^2 \leq \frac{2}{\eta T} (\mathcal{L}(\mathbf{w}_1) - \mathbb{E}[\mathcal{L}(\mathbf{w}_{T_w})]) + \frac{T_w L \eta \sigma^2 \mathcal{O}(r)}{T} + \mathcal{O}(\mu^2), \quad (9)$$

and for the post-warm-up phase, Equation (7) and (8) with Assumption 4.1 yields

$$\begin{aligned} &\frac{1}{T} \sum_{t=T_w+1}^T \|\nabla \mathcal{L}(\mathbf{w}_t)\|_2^2 \\ &\leq \frac{s_u}{T} \sum_{t=T_w+1}^T \|\nabla \mathcal{L}(\mathbf{w}_t)\|_{\Sigma_t^{-1}}^2 \\ &\leq \frac{2s_u}{\eta T} (\mathbb{E}[\mathcal{L}(\mathbf{w}_{T_w+1})] - \mathbb{E}[\mathcal{L}(\mathbf{w}_T)]) + \frac{s_u(T - T_w)L\eta(\mathcal{O}(\bar{\beta}_1^2 L^2 \eta^2) + \sigma^2)(\text{tr}\Sigma_t^{-1} + 2s_l^{-1})}{T} + \mathcal{O}(\mu^2). \end{aligned} \quad (10)$$

Take $s = \max\{1, s_u\}$, combine Equation (9) and (10),

$$\begin{aligned} e &\leq \frac{2s}{\eta T} (\mathbb{E}[\mathcal{L}(\mathbf{w}_1)] - \mathbb{E}[\mathcal{L}(\mathbf{w}_T)]) + \frac{T_w \eta L \sigma^2 \mathcal{O}(r)}{T} + s L \eta (\mathcal{O}(\bar{\beta}_1^2 L^2 \eta^2) + \sigma^2) (\text{tr}\Sigma_t^{-1} + 2s_l^{-1}) + \mathcal{O}(\mu^2) \\ &\leq \frac{L\sigma^2}{\sqrt{T}} + \frac{2}{T\sqrt{T}} + \frac{T_w(\sigma^2 + \mathcal{O}(\bar{\beta}_1^2 L^2 \eta^2))}{T\sqrt{T}} + \mathcal{O}(\mu^2). \end{aligned}$$

We omit the higher order terms to arrive at the target. $\square$